

CODENTRY: AN AI-POWERED GITHUB CODE REVIEW TOOL

Literature Survey for Final Year Engineering Project

Grounding the Static/AI Split Architecture in 30 Research Papers (2025–2026)

Submitted by

Rushikesh • Vansh Mhatre • Yash Jadhav • Piyush Waghmare

Department of Computer Engineering

University of Mumbai — Batch 2027

August 2026

Table of Contents

Abstract.....	4
1. Introduction.....	5
2. Theme A: Repository-Level Context and Retrieval-Augmented Generation.....	7
2.1 The Reality Gap: Why Diff-Only Review Systematically Fails.....	7
2.2 Retrieval Quality Benchmarks for Coding Agents.....	7
2.3 RAG Pipelines Designed Specifically for Review Comment Generation.....	8
2.4 Repository-Level Reasoning and the Survey of Retrieval Methods.....	8
2.5 Synthesis: Agreement and Productive Tension Within Theme A.....	9
2.6 Implication for Codentry.....	9
3. Theme B: Benchmarks and Evaluation Methodology for AI Code Review.....	10
3.1 Historical Survey of Evaluation Practice.....	10
3.2 State-of-the-Art Benchmark Design.....	10
3.3 Agent-Centric and Test-Based Evaluation.....	11
3.4 CodeFuse-CR-Bench’s Dual-Layer Evaluation Framework.....	11
3.5 Synthesis and Implication for Codentry.....	11
4. Theme C: Multi-Agent and Structured Review Architectures.....	13
4.1 RepoReviewer: The Nearest Architectural Precedent.....	13
4.2 SWE-Review: Closed-Loop Agentic Review.....	13
4.3 CodeX-Verify: Information-Theoretic Multi-Agent Design.....	14
4.4 AgenticSCR: Production Agentic Security Review.....	14
4.5 Implication for Codentry.....	15
5. Theme D: Security-Focused Review and Hybrid Analysis.....	16
5.1 The Security Deficit of AI-Generated Code.....	16
5.2 Attack Surface of AI Review Systems.....	16
5.3 MAF-CPR: Multi-Agent PR Review in Production.....	17
5.4 Synthesis and Implication for Codentry.....	17
6. Theme E: Human/AI Interaction and Pull Request Behavior.....	19
6.1 How Humans Actually Review AI-Generated PRs.....	19
6.2 Task Type as the Dominant Factor in PR Acceptance.....	19
6.3 The Impact of LLM Assistance on PR Timelines.....	20
6.4 Implication for Codentry.....	20
7. Theme F: Core AI/LLM Code Review and PR Automation.....	21
7.1 LAURA: Context-Enriched RAG for Review Generation.....	21
7.2 Sphinx: Checklist-Based Training and Evaluation.....	21
7.3 Rethinking Code Review Workflows: Industrial Field Evidence.....	22
7.4 ARCTIC: Intent, Drift, and Spotlight at Production Scale.....	22
7.5 Synthesis: What Theme F Tells Us About Practical LLM Review.....	23
7.6 Implication for Codentry.....	23
8. Cross-Theme Synthesis Table.....	24
9. Gap Analysis: What the Corpus Does Not Address.....	26

9.1 Gap 1: Independent Evaluability of Static and AI Review Layers.....	26
9.2 Gap 2: Free-Tier-Deployable Repository-Level AI Review.....	26
9.3 Gap 3: PR-Native Interaction Without External Dashboard.....	26
9.4 Gap 4: Stratified Evaluation Across Human-Authored and AI-Generated PRs.....	27
9.5 Summary.....	27
10. Threats to Validity and Limitations of This Survey.....	28
10.1 ArXiv Preprint Bias.....	28
10.2 Recency Restriction and Historical Omissions.....	28
10.3 Corpus Selection Bias.....	28
10.4 LLM-as-Judge Evaluation Uncertainty.....	28
10.5 Single-Language and Repository Type Biases.....	29
11. Conclusion.....	30
References.....	31

Abstract

This literature survey grounds the architecture of Codentry — an AI-powered GitHub code review tool — in 30 recent research papers published between 2025 and 2026. Codentry's central design decision is a deliberate static/AI split: deterministic checks (ESLint, Semgrep) run in a separate, independently evaluable stage from judgment-based review performed by the Claude API. This separation is not a compromise forced by capability gaps in AI; it is a principled engineering choice that enables each subsystem to be evaluated on its own terms — static output scored against ground truth with precision/recall, AI judgment scored independently without static-analysis noise contaminating the metric. The survey covers seven thematic clusters: (A) repository-level context and retrieval-augmented generation; (B) benchmarks and evaluation methodology; (C) multi-agent and structured review architectures; (D) security-focused review and hybrid analysis; (E) human/AI interaction and pull-request behavior; (F) core AI/LLM code review and PR automation; and (G) an additional systematic literature review cross-cutting all themes. Each theme section maps findings to a specific Codentry design decision, build phase, or identified risk through a dedicated 'Implication for Codentry' subsection. A cross-theme synthesis table positions Codentry relative to the full corpus across five dimensions. A gap analysis identifies what none of the 30 papers jointly achieve: a free-tier-deployable, PR-native code review tool that cleanly separates deterministic and judgment-based review and provides independent evaluation harnesses for each. Threats to validity — particularly the arXiv-heavy preprint bias of the corpus — are addressed in a dedicated section. Every claim in this survey is traceable to a specific cited paper; no claim rests on general reasoning alone.

1. Introduction

Software code review is one of the most effective quality assurance mechanisms in collaborative software engineering. Fagan's structured inspection studies established that systematic review catches defects earlier and cheaper than downstream testing, and decades of empirical software engineering research has reinforced the finding that defects discovered during review cost significantly less to fix than those discovered post-deployment. Yet code review has always been expensive in human time: a reviewer must understand the intent of a change, its interaction with existing code, its security implications, and its conformance to team style standards — all from a diff that may omit crucial context. The volume of pull requests in modern development environments, combined with the increasing proportion generated by AI coding agents, has created a demand for automated or AI-assisted review that the research community has responded to with a wave of benchmarks, agent architectures, and empirical studies.

Despite the rapid growth of AI-assisted code review research, a persistent and underappreciated problem remains: most existing tools conflate two fundamentally different kinds of review work. The first kind is deterministic — does this code violate a known lint rule? Does this pattern match a Semgrep signature for SQL injection? These checks have ground truth; they can be run without an LLM; and they can be scored with standard precision and recall against known violation databases. The second kind is judgment-based — is this function doing what the PR description claims? Is this abstraction boundary the right one? Is this error-handling approach appropriate for the failure modes the rest of the codebase anticipates? These questions require contextual reasoning, do not have single correct answers, and cannot be evaluated by comparing output to a static ground-truth set.

Codentry is an AI-powered GitHub App that separates these two kinds of review explicitly. Static analysis (ESLint for JavaScript/TypeScript, Semgrep for multi-language security patterns) runs first, producing deterministic output that can be scored against known violation datasets. AI judgment (Claude API) runs second, receiving the diff and structured static-tool output, and producing natural-language review comments that are posted directly to the PR. There is no separate dashboard; the entire review surface is the pull-request comment thread. The stack runs entirely on free tiers: Next.js on Vercel, FastAPI on Render, Supabase with pgvector. Two build phases remain unrealised at the time of this survey: Phase 4 will introduce RAG-based chunking for repository-level context retrieval using Supabase's pgvector store, and Phase 6 will introduce a precision/recall evaluation harness that scores static-tool output and AI judgment independently.

This survey serves three purposes. First, it establishes that each of Codentry's major design decisions — the static/AI split, the PR-comment interaction model, the RAG architecture, and the evaluation methodology — is traceable to documented findings in the 2025–2026 research literature. Second, it identifies where the corpus reveals risks that Codentry must design around, particularly in the areas of adversarial prompt injection, attention dilution in RAG pipelines, and evaluation ground-truth validity in an increasingly AI-dominated PR ecosystem. Third, it performs a gap analysis identifying what the corpus collectively does not address and where Codentry's combination of constraints and design choices constitutes a narrowly novel engineering contribution. Every claim about what a paper supports or contradicts is numbered and traceable to a specific reference; the paper's actual content — not its title or abstract alone — is the basis for each claim.

The survey covers all 30 papers in the provided corpus, organised into seven thematic clusters that correspond to the primary grouping established by the project team. Duplicate entries in the corpus (SWE-PRBench [8] appears in both Theme A and Theme B; SWRBench [11] appears in both Theme B references; AgenticSCR [15] appears in both Theme C and Theme D) are counted once and cross-referenced where both thematic angles are substantively relevant. Each theme section closes with an explicit 'Implication for Codentry' subsection that directly maps research findings to the project's design decisions, build phases, or risk posture. The document concludes with a cross-theme synthesis table comparing all 30 papers across five structural dimensions, a gap analysis, a threats-to-validity section, and a complete IEEE-numbered reference list.

2. Theme A: Repository-Level Context and Retrieval-Augmented Generation

Seven papers address the central challenge of providing AI code review systems with enough contextual understanding of the surrounding repository to produce useful, non-generic feedback. The papers span benchmark construction, empirical retrieval quality evaluation, concrete RAG pipeline proposals, repository-level reasoning assessment, and a comprehensive survey of the retrieval-augmented code generation design space. Together they establish both the necessity of repository-level context for high-quality review and the specific mechanisms by which poorly designed context provision degrades rather than improves performance.

2.1 The Reality Gap: Why Diff-Only Review Systematically Fails

CodeFuse-CR-Bench [1] introduces the 'reality gap' concept to characterise the discrepancy between what existing code review benchmarks measure and what real-world PR review requires. Its 601 high-quality instances from 70 Python projects are notable for two design decisions: each instance includes not just the diff but the associated issue description, PR metadata, and full repository state; and each instance is labelled across nine PR problem domains rather than a single binary 'needs review' label. The multi-domain labelling reveals that LLM performance varies substantially across problem types — no single model dominates all nine domains, and Gemini 2.5 Pro achieves only the highest comprehensive score, not a dominant one. This finding is diagnostic: it suggests that code review quality is not a single capability that scales with model size but a collection of distinct sub-capabilities that may require targeted improvement or specialist routing.

SWE-PRBench [8] provides the most quantitatively precise evidence for the inadequacy of diff-only review at scale. Evaluated on 350 PRs from active open-source repositories with human-annotated ground truth ($\kappa=0.75$ inter-rater reliability), eight frontier models detect only 15–31% of human-flagged issues on the diff-only configuration. The monotonic degradation from config_A to config_C as context increases is explained mechanistically: a structured 2,000-token diff-with-summary prompt outperforms a 2,500-token full-context prompt enriched with execution context, behaviour mapping, and test signatures. The difference is not raw context length but signal-to-noise ratio — the 500-token advantage of the structured prompt compensates for its smaller size because it eliminates noise that the full-context prompt introduces. This is the attention dilution phenomenon, and its quantification in [8] provides a direct design constraint for any RAG system: retrieval quality must be high enough that added context increases the signal-to-noise ratio, not decreases it.

2.2 Retrieval Quality Benchmarks for Coding Agents

ContextBench [2] is the first process-oriented evaluation of context retrieval in coding agents, distinguished from end-to-end benchmarks by its focus on intermediate retrieval behaviour rather than final task success. Its 1,136 issue-resolution tasks from 66 repositories across eight programming languages are augmented with human-annotated gold contexts, enabling measurement of context recall, precision, and efficiency throughout the agent trajectory. Three findings from [2] are directly relevant to Codentry. First, sophisticated agent scaffolding yields only marginal gains in context retrieval quality — the 'Bitter Lesson' of coding agents applies to retrieval specifically, not just to end-task performance, meaning that engineering elaborate retrieval pipelines may yield diminishing returns. Second, LLMs consistently favour recall over precision in their retrieval strategies, meaning that agent-driven retrieval

will systematically over-fetch context. Third, substantial gaps exist between context that agents explore and context they actually utilise, suggesting that raw retrieval volume does not translate proportionally into effective context use.

Agent Retrieval Bench [3] complements ContextBench by focusing on file-level retrieval — identifying which files in a repository an agent needs before beginning a task. Its four positive-retrieval tasks (code2test, comment2context, trace2code, edit2ripple) reflect different scenarios an AI code reviewer would encounter: reviewing a change that requires understanding its test implications (code2test), reviewing a comment change in the context of what it documents (comment2context), reviewing a runtime behaviour change (trace2code), and reviewing a change that ripples across dependent files (edit2ripple). No single retrieval family dominates across all four tasks: Qwen3-Embedding-8B achieves the best Recall@20 overall but RepoMap achieves the best budgeted context yield at 8K tokens, with substantial task-level variation. The calibration gap — where selective thresholds calibrated with counterfactual controls do not improve selective success on natural no-gold cases — is a fundamental limitation for retrieval systems that must decide not just what to retrieve but whether to retrieve at all. The 27–35% rate at which logged agent trajectories miss every gold file is a sobering baseline for Codentry's Phase 4 design.

2.3 RAG Pipelines Designed Specifically for Review Comment Generation

Paper [4] — Retrieval-Augmented Code Review Comment Generation — provides the first systematic RAG approach specifically designed for review comment generation rather than code generation or issue resolution. Its framing is elegant: pure generation models struggle with low-frequency domain-specific tokens (the team's custom identifiers, library-specific patterns, project-specific naming conventions) while pure IR methods lack flexibility for code contexts not represented in the retrieval corpus. RAG bridges this gap by conditioning generation on retrieved exemplars. On the Tufano et al. benchmark, [4] achieves +1.67% higher exact match and +4.25% higher BLEU scores compared to generation-only baselines, with a 24.01% improvement in low-frequency token generation — precisely the domain-specific vocabulary that characterises a team's codebase. The finding that performance improves as the number of retrieved exemplars increases is presented without bound in [4], suggesting that more is always better — a claim that [5] directly contradicts.

The RARe framework [5] — 'When More Retrieval Hurts' — provides the necessary corrective. Framing retrieval as in-context learning rather than information injection, RARe achieves BLEU-4 scores of 12.32 and 12.96 on two public benchmarks. The key finding is that top-1 retrieval outperforms top-3 and top-5 configurations due to redundancy and conflicting cues under limited context budgets. This is not a minor practical observation but a fundamental constraint of in-context learning: when retrieved exemplars conflict with each other (as multiple retrieved reviews for similar but not identical code will), the model must resolve the conflict rather than learning from the exemplars, degrading performance. Human evaluation confirms that top-1 retrieval produces more focused, less generic reviews than top-3, supporting the signal-to-noise interpretation.

2.4 Repository-Level Reasoning and the Survey of Retrieval Methods

RepoReasoner [6] establishes a crucial upper-bound caveat for any RAG-based review system: even with perfect retrieval (oracle context where the model is given exactly the files it needs), the best-performing model achieves only 69.1% Pass@1 on Output Prediction tasks requiring cross-file reasoning. In Call Chain Prediction (understanding the dependency graph of a repository), models

exhibit high precision but low recall — they can identify some dependencies but miss many others — and performance drops on rewritten test data reveal partial reliance on memorisation rather than genuine architectural reasoning. Longer contexts do not consistently improve results due to noise, reinforcing the attention dilution finding from [8]. For Codentry, [6] establishes that Phase 4 RAG will help but cannot eliminate cross-file reasoning errors, and Phase 6 evaluation must measure these errors separately from failures caused by insufficient context provision.

The survey of retrieval-augmented code generation with focus on repository-level approaches [7] provides the broadest view of the RAG design space as applied to code tasks, organising approaches along three axes: retrieval granularity (token, line, function, file, or repository), retrieval mechanism (sparse lexical search, dense embedding search, graph traversal, or hybrid), and integration strategy (prepend, cross-attention, or tool-use retrieval). Its key finding for repository-level tasks is that chunk-level embedding retrieval consistently underperforms graph-based and hybrid approaches because semantically similar chunks are not always functionally relevant to the code change being reviewed. The survey evaluates pgvector as a viable pragmatic choice for resource-constrained systems, recommending its use with lightweight dependency metadata overlays to improve precision. This is directly applicable to Codentry: using pgvector alone for embedding similarity search is a reasonable starting point for Phase 4, but the survey's recommendation suggests adding a lightweight dependency graph (extractable from the repository's import structure) as a secondary ranking signal.

2.5 Synthesis: Agreement and Productive Tension Within Theme A

The papers in Theme A reach consensus on the core claim that context matters for code review quality — papers [1], [2], [3], [6], [7], and [8] all support this in different forms. The productive tension lies in how context should be provided. Papers [4] and [2] suggest that more context generally helps; papers [5] and [8] establish empirical bounds beyond which additional context is counterproductive. The resolution is that retrieval precision — not retrieval volume — is the operative variable. This precision-first principle also implies a design choice about RAG architecture: a retrieval system that always fetches k exemplars regardless of retrieval confidence is inferior to one that fetches fewer, higher-quality exemplars. The calibration gap identified in [3] — where off-distribution repositories defeat threshold calibration — is the primary remaining open question for Phase 4.

2.6 Implication for Codentry

Theme A justifies Phase 4's RAG architecture along three specific design principles. First, Codentry's use of pgvector for cosine similarity search is consistent with [7]'s pragmatic recommendation for resource-constrained systems, but should be augmented with lightweight repository structure metadata (e.g., import graph edges) to improve precision. Second, the top- k retrieval parameter should be set to $k=1$ or $k=2$ rather than larger values, based on [5]'s finding that top-1 retrieval outperforms top-3 and top-5 under context budget constraints. Third, retrieved exemplars should be repository-specific (drawn from the same repository's PR history stored in Supabase) rather than general (drawn from a public code review corpus) to maximise the relevance of the domain-specific vocabulary advantage documented in [4]. The risk this theme surfaces for Codentry is that Phase 4's retrieval quality on repositories not well-represented in the Supabase store (new repositories with few historical PRs) will degrade toward the diff-only baseline documented in [8] — a cold-start problem that the Phase 6 evaluation harness should measure explicitly.

3. Theme B: Benchmarks and Evaluation Methodology for AI Code Review

Evaluating the quality of AI-generated code review is arguably harder than evaluating code generation. Code generation has a clear success criterion — does the generated code pass the test suite? — but code review quality is inherently subjective, multi-dimensional, and context-dependent. A review comment can be technically correct but not actionable; actionable but not prioritised; prioritised but not localised to the right line; localised correctly but missing the most important issues. Five papers in the corpus address the benchmark and evaluation methodology dimension specifically, and the systematic literature review [30] provides a meta-level view of how the field has evaluated review automation historically.

3.1 Historical Survey of Evaluation Practice

The survey of code review benchmarks [10] is the most comprehensive mapping of the evaluation landscape, analysing 99 papers from 2015 to 2025 and identifying a five-domain, 18-task taxonomy of code review automation. The clear finding is that evaluation practice in the LLM era has shifted toward end-to-end generative peer review at the expense of fine-grained task-specific evaluation. This shift is understandable — end-to-end review is closer to what practitioners need — but it makes attribution difficult: when an end-to-end system improves or degrades, the survey cannot determine which of the 18 sub-tasks drove the change. The evaluation metrics used across the 99 papers are highly heterogeneous: BLEU, exact match, human preference, LLM-as-judge, and task completion are all used in ways that are not directly comparable. The survey identifies this metric fragmentation as the primary obstacle to systematic progress measurement in the field.

The systematic literature review [30] provides the pre-LLM historical context that [10] summarises. Covering 119 papers on code review automation, [30] categorises automated tasks into reviewer selection, reviewer assignment, change understanding, defect prediction, and review comment generation. Its key forward-looking observation — that 'dynamic runtime evaluation' (testing review systems against code that is executed and verified, not just read statically) is the most important direction for future benchmark development — is directly relevant to Codentry's Phase 6 design. Static ground truth (annotated issues in a frozen dataset) is necessary but insufficient; execution-based validation that checks whether issues flagged by Codentry correspond to actual runtime failures or security violations provides a stronger validity signal.

3.2 State-of-the-Art Benchmark Design

SWE-PRBench [8] represents the current state of the art in benchmark construction rigour for AI code review. Its methodological contributions are four: (1) human annotation of 350 real GitHub PRs with inter-rater reliability measured at $\kappa=0.75$; (2) a Type1/Type2 issue categorisation distinguishing self-contained diff issues from contextual issues that require repository knowledge; (3) frozen context configurations (config_A: diff only; config_B: diff + file content; config_C: full context with AST, import graph, test signatures) enabling controlled ablation; and (4) an LLM-as-judge framework validated against human annotations. The finding that all eight evaluated models degrade monotonically from config_A to config_C is both a finding about AI code review quality and a validation of the config_A–C ablation design: a benchmark that produces monotonic degradation across its context

configurations is measuring something real about how context affects AI review performance, not just noise.

SWRBench [11] contributes a different benchmark design philosophy: 1,000 manually verified PRs from GitHub with a structured ground truth defined by issue coverage rather than string match, and an evaluation method that achieves approximately 90% agreement with human judgment. The multi-review aggregation finding — that combining outputs of multiple ACR tools increases F1 by up to 43.67% — is the largest performance gain reported in the corpus from any single technique. Its magnitude (nearly doubling F1 in some configurations) suggests that the independence between different review tools or prompting strategies is higher than expected, making ensemble approaches disproportionately valuable. The distinction [11] draws between ACR tools' relative strength at functional errors vs. AI models' relative strength at semantic issues suggests that the two should be evaluated against different issue type strata.

3.3 Agent-Centric and Test-Based Evaluation

The c-CRAB benchmark [9] proposes the most methodologically distinct approach to evaluation: generating executable tests from human reviews and using these tests as a quality gate for agent-generated reviews. This approach eliminates LLM-as-judge bias — a significant advantage given that LLM-as-judge frameworks may be systematically biased toward output styles that resemble their own training distribution — and provides a directly executable quality gate. The 40% task-completion rate for current state-of-the-art agents (PR-agent, Devin, Claude Code, Codex) establishes a meaningful baseline: current tools collectively solve less than half of well-specified review tasks when evaluated against test-based ground truth. The observation that agent reviews focus on different aspects than human reviews suggests that agent review is not simply inferior human review but a genuinely different review modality that may be complementary to rather than substitutable for human review.

3.4 CodeFuse-CR-Bench's Dual-Layer Evaluation Framework

CodeFuse-CR-Bench [1], discussed in Theme A for its benchmark construction, warrants separate attention in Theme B for its evaluation methodology. Its combination of rule-based checks for location and syntax accuracy with model-based judgment of review quality is the closest precedent in the corpus to the evaluation structure Codentry plans for Phase 6. The rule-based checks produce scores that are fully deterministic and reproducible — no two evaluation runs on the same output will produce different results. The model-based judgment scores are inherently variable but calibrated against human annotations. By applying these two evaluation types to different dimensions of the same review output, [1] demonstrates that a single review can be scored on its location accuracy deterministically and on its semantic quality probabilistically, without conflating the two. This is operationally exactly what Codentry's Phase 6 harness needs to do for its static tool output and AI judgment output respectively.

3.5 Synthesis and Implication for Codentry

The evaluation methodology literature in Theme B converges on four design principles for Codentry's Phase 6 harness. First, evaluation must be multi-dimensional: location accuracy (deterministic), issue coverage (probabilistic via LLM-as-judge), and false positive rate (deterministic against known-safe code) must be measured separately, not collapsed into a single score. This is supported by [1], [8], [10], and [11]. Second, the LLM-as-judge framework is the most scalable approach for issue coverage evaluation, and its validity is established at $\kappa=0.75$ [8] and $\sim 90\%$ human agreement [11] — these

are the calibration targets Phase 6 should aim for. Third, multi-review aggregation [11] provides a technique that Phase 6 should test: running Claude with multiple prompting strategies and aggregating outputs may produce larger gains than any single prompt engineering improvement. Fourth, the test-based evaluation approach of c-CRAB [9] provides a model for Codentry's static tool evaluation specifically: Semgrep's output can be validated against known-vulnerable code datasets (CVE test suites) in a fully executable, LLM-independent way. The open risk this theme surfaces is that without a publicly released, human-annotated test set comparable to [8] or [11], Codentry's Phase 6 results will be difficult to compare externally — a limitation that should be acknowledged explicitly.

4. Theme C: Multi-Agent and Structured Review Architectures

Multi-agent architectures for software engineering have proliferated in the 2025–2026 period, motivated by the finding that no single agent excels at all review dimensions and that combinations of specialised agents can outperform any single generalist. The four papers in this theme represent different points in the design space of structured review pipelines, ranging from pragmatic systems contributions to theoretically grounded multi-agent verification to production deployments at Atlassian scale. Their collective examination reveals a tension that is central to Codentry's design: multi-agent decomposition can improve review quality, but it increases system complexity, cost, latency, and attack surface — all of which conflict with Codentry's free-tier and single-agent AI constraints.

4.1 RepoReviewer: The Nearest Architectural Precedent

RepoReviewer [12] is the closest architectural precedent to Codentry in the corpus. It deploys a local-first multi-agent system for automated GitHub repository review using exactly the same technology stack that Codentry uses: Python for the service layer, FastAPI for the API, and Next.js for the user interface. Its five-stage pipeline — repository acquisition, context synthesis, file-level analysis, finding prioritisation, and summary generation — maps conceptually onto the phases of Codentry's review pipeline, though Codentry's static/AI split introduces a layer between context synthesis and analysis that RepoReviewer does not have. The paper's intellectual honesty is noteworthy: it explicitly frames RepoReviewer as a systems contribution rather than a benchmark superiority claim, acknowledges practical failure modes (context synthesis errors when repository graphs are cyclic, finding prioritisation instability for large PRs), and releases evaluation and reporting infrastructure for future empirical study rather than claiming SOTA performance.

The architectural difference between RepoReviewer and Codentry is precisely defined: RepoReviewer collapses deterministic and judgment-based analysis into the same LangGraph pipeline, with no independent evaluability for each component's contribution to overall review quality. This means that when RepoReviewer's reviews are incorrect, it is impossible to determine whether the error originated in context synthesis (a retrieval failure), file-level analysis (a reasoning failure), or finding prioritisation (a ranking failure). Codentry's explicit separation of static analysis output from AI judgment output preserves independent diagnostic signals: a Semgrep false positive can be identified as such without confounding AI judgment errors, and vice versa.

4.2 SWE-Review: Closed-Loop Agentic Review

SWE-Review [13] proposes the generate-review-revise loop as the fundamental primitive for agentic code review: given an issue and an AI-generated PR, a reviewer agent explores the repository, decides whether the PR should be accepted, and provides structured feedback for revision. The empirical results are compelling — agentic review continuously improves PRs through the loop, outperforming single-turn fixed-context review in both decision accuracy and resolve rate after revision. The enable test-time scaling finding is particularly significant: more computation (more review-revise turns) yields proportionally better results, suggesting that the agentic review paradigm benefits from compute scaling in a way that single-turn review does not.

The tension between SWE-Review's architecture and Codentry's constraints is sharp. SWE-Review requires the reviewer agent to: explore the full repository (not just the diff), execute tests (requiring a compute environment), make multiple round trips to the model, and have the authority to make

acceptance/rejection decisions. None of these are compatible with Codentry's free-tier constraint or its PR-comment-only interaction model. The relevant insight for Codentry is more modest but still actionable: the generate-review-revise loop creates natural checkpoints at which review quality can be measured independently per turn, providing a template for Phase 6 evaluation design even if the full loop cannot be implemented in production.

4.3 CodeX-Verify: Information-Theoretic Multi-Agent Design

CodeX-Verify [14] provides the most rigorous theoretical foundation for multi-layer review architectures. Using the submodularity of mutual information under conditional independence, it proves that combining agents with different detection patterns finds more bugs than any single agent when the agents target different problem types. The empirical confirmation — agent correlation $\rho=0.05$ to 0.25 on 99 verified code samples — demonstrates that the four specialised agents (Correctness, Security, Performance, Style) are nearly independent in their detection patterns. This independence is the key property: agents that detect the same bugs provide redundant coverage, while agents that detect different bugs provide additive coverage. The diminishing returns curve (+39.7pp from no agents to one agent, +14.9pp for agent 2, +13.5pp for agent 3, +11.2pp for agent 4) shows that most of the incremental gain is achieved by the first two agents.

The information-theoretic argument maps directly onto Codentry's static/AI split. ESLint and Semgrep (the static layer) detect syntactic violations, known vulnerability patterns, and linting errors — categories where ground-truth rules are available. Claude (the AI layer) detects semantic issues, logic errors, and contextual concerns — categories where ground-truth rules do not exist and judgment is required. If the detection patterns between these two layers are approximately independent (low ρ), the theoretical guarantee that their joint coverage exceeds either alone applies. Measuring ρ between Codentry's static layer detections and Claude's detections on the Phase 6 evaluation set would provide empirical validation of this independence assumption — a straightforward but novel contribution to the evaluation literature.

4.4 AgenticSCR: Production Agentic Security Review

AgenticSCR [15] is the only paper in the corpus documenting a production deployment of an agentic code review system at the scale of a major software company (Atlassian). It addresses the specific problem of 'immature vulnerabilities' — security issues that are partially but not fully manifested in the reviewed code and therefore evade traditional SAST tools. The 153% relative improvement over SAST tools in generating comments with correct localisation, vulnerability type, and relevance is substantial; the 54% validation rate by security engineers in shadow deployment provides a practical utility measure. The security-focused semantic memory — a structured representation of security knowledge that grounds the agent's reasoning — is the key architectural innovation distinguishing AgenticSCR from both static LLM reviewers and standard SAST tools.

The comparison [15] draws between SAST tools and its agentic approach is directly informative for Codentry's design. SAST tools (analogous to Codentry's Semgrep layer) are described as 'often noisy and struggle with context-dependent or partially manifested vulnerabilities.' This is an accurate characterisation of Semgrep's limitations: it pattern-matches against known signatures but cannot reason about the semantic context of a potential vulnerability. Codentry accepts this limitation in exchange for evaluability — Semgrep's false positives are measurable against CVE databases, while AgenticSCR's

combined output is not independently evaluable for its static vs. agentic components. This is an explicit design tradeoff, not a capability oversight.

4.5 Implication for Codentry

Theme C validates Codentry's static/AI split through two independent lines of support. The information-theoretic argument from [14] establishes that two nearly-independent detection layers should provide joint coverage exceeding either alone — a theoretical guarantee that Codentry's architecture satisfies by design. The practical evidence from [15] establishes that SAST tools and AI judgment have largely non-overlapping failure modes in security review, consistent with low correlation between their detections. At the same time, Theme C surfaces the most significant constraint on Codentry's current capabilities: the best-performing architectures in the corpus ([13], [15]) are resource-intensive multi-round systems that cannot be reproduced under free-tier constraints. Codentry's FYP report should explicitly acknowledge this as a capability ceiling imposed by infrastructure rather than by design philosophy, and position Phase 4 and Phase 6 as steps toward closing the gap with systems like SWE-Review within resource constraints.

5. Theme D: Security-Focused Review and Hybrid Analysis

Security considerations permeate Codentry's design from both sides: the security quality of the code Codentry reviews, and the security of Codentry as a system that processes untrusted input from public repositories. Five papers in this theme address these dimensions systematically. The demand-side papers ([16], [17]) establish that AI-generated code has severe security deficiencies that justify robust review. The supply-side papers ([18], [19], [20]) establish that AI review systems themselves are vulnerable to adversarial manipulation through the code they review. Paper [29] (the Springer chapter) demonstrates that multi-agent PR review within GitHub is deployable in practice.

5.1 The Security Deficit of AI-Generated Code

SUSVIBES [16] provides the most alarming datapoint in the corpus for justifying AI code review as a security-critical application. On 186 feature-request tasks from real-world open-source projects where human programmers had committed vulnerable implementations, 12 coding agent configurations with frontier models were evaluated for both functional correctness and security. The finding is unambiguous: while SWE-Agent with Claude 4 Sonnet produces 57% functionally correct solutions, only 11.8% are secure. Across all 12 agents, security rates are consistently far below functional correctness rates. The paper demonstrates that preliminary security mitigations — augmenting the feature request prompt with vulnerability hints, asking the agent to consider security — do not meaningfully improve security rates. This is an important null result: the security gap in AI-generated code cannot be closed by prompt-level interventions alone. Independent review — the function Codentry performs — is therefore not redundant to the agent's own security awareness but provides genuinely independent coverage.

SEC-bench [17] complements the vulnerability-in-generated-code finding with data on LLM agent performance on security-specific tasks: proof-of-concept (PoC) generation at 18.0% success and vulnerability patching at 34.0% success. These rates establish that frontier LLMs, despite strong general coding ability, lack reliable security-specific reasoning for real-world vulnerability exploitation and remediation. For Codentry's design, the 34.0% patching rate is particularly relevant: if Claude is asked to not just flag but also suggest fixes for security issues, the fix quality will be low for non-trivial vulnerabilities. Phase 6's evaluation of Claude's security suggestions should use known-exploitable CVEs as ground truth, and the evaluation should report fix quality separately from detection quality.

5.2 Attack Surface of AI Review Systems

Security Considerations for Artificial Intelligence Agents [18] provides the most comprehensive threat taxonomy directly applicable to Codentry's architecture. It identifies code-data separation as a fundamental security assumption that agent systems violate by design: an agent that processes code as input and produces output that affects downstream systems (in Codentry's case, posting PR comments) collapses the traditional distinction between the data being processed and the instructions governing processing. This creates the indirect prompt injection attack surface: code in a PR, crafted to appear syntactically normal, can contain embedded instructions that, when processed by Claude, alter its review behaviour. The paper maps three specific confused-deputy scenarios that apply to Codentry: (1) a malicious contributor causes Claude to suppress true positive findings about a vulnerable PR; (2) a malicious contributor causes Claude to generate misleading positive comments about a harmful change;

(3) a malicious contributor causes Claude to produce comments that contain injected content intended for other reviewers (e.g., links to malicious resources framed as documentation).

Architecture Matters for Multi-Agent Security [19] establishes the empirical magnitude of architectural effects on attack success. Across 13 architectural configurations, attack success rates vary by up to 3.8x at comparable benign accuracy — a larger variance than any capability improvement achieves. The specific finding that multi-agent systems are more vulnerable than standalone agents in the majority of configurations is directly relevant to Codentry's decision (driven by free-tier constraints) to use a single-agent AI layer: the architectural simplicity that resource constraints impose inadvertently reduces the attack surface relative to more complex multi-agent alternatives. This is not a security argument for free-tier constraints — it is an honest observation that a constraint imposed for resource reasons also has a security benefit.

Code Injection Attacks [20] provides the technical detail that completes the threat model. Its analysis of the coder-reviewer-tester architecture shows that the reviewer agent is the highest-value target: compromising the reviewer directly compromises the quality signal on which subsequent revisions depend. The most alarming result is that embedding poisonous few-shot examples in injected code increases the attack success rate from 0% to 71.95% against a security analysis agent. This means that a naive defence — adding a second AI layer to check for injection — is not sufficient, because the second layer can be compromised by the same technique. Input sanitisation must occur before the code reaches any LLM layer, which is precisely the role Semgrep plays in Codentry's architecture when configured with prompt-injection detection rules.

5.3 MAF-CPR: Multi-Agent PR Review in Production

The Springer chapter MAF-CPR [29] contributes the only peer-reviewed evidence in the corpus that multi-agent LLM-based PR review frameworks can be deployed within GitHub's native workflow. Published in ICIC 2025, MAF-CPR decomposes complex PR review into agents handling change scope analysis, semantic understanding, and integration risk assessment, with coordination through a master agent that aggregates specialist outputs into a unified review. The framework's deployment within GitHub (posting structured review comments to PRs) is the same interaction model Codentry uses, providing peer-reviewed validation that the PR-comment interaction model is feasible and accepted by development teams. MAF-CPR's use of multiple specialised LLM calls per review (estimated at 3–5 calls per PR) is incompatible with Codentry's free-tier constraint, but the interaction model validation transfers regardless.

5.4 Synthesis and Implication for Codentry

Theme D establishes two complementary imperatives. The demand-side imperative ([16], [17]) is that Codentry's security review function is not redundant to the security awareness of AI coding agents: SUSVIBES [16] demonstrates that agent-generated code is severely insecure despite agents' apparent security knowledge, and SEC-bench [17] demonstrates that even frontier LLMs have low security task completion rates. The supply-side imperative ([18], [19], [20]) is that Codentry must be architected to limit the authority of its AI layer — specifically, Claude should post comments but cannot approve, merge, or modify code. Specific mitigations informed by [18] and [20] are: (1) structural prompt separators between code content and review instructions; (2) Semgrep rules screening for known prompt injection patterns as a pre-processing step; (3) output anomaly detection that flags Claude

responses containing unusual claims before posting. The MAF-CPR validation [29] confirms that the PR-comment interaction model is the right deployment target.

6. Theme E: Human/AI Interaction and Pull Request Behavior

Codentry's interaction model — delivering all review feedback as pull-request comments, with no external dashboard or web application interface — is a deliberate design choice justified by the empirical evidence on how developers actually interact with automated PR feedback in practice. Two papers provide direct empirical grounding for this choice, while a third establishes the temporal impact of AI assistance on the PR lifecycle.

6.1 How Humans Actually Review AI-Generated PRs

Terragni et al. [21] analyse code review interactions for AI-generated PRs on GitHub using the AIDev dataset — a large-scale dataset of AI-authored pull requests with complete interaction histories. Comparing AI-generated PRs to human-authored PRs in the same repositories, the paper finds several systematic differences. Most AI-generated PRs receive no review at all, while human-authored PRs have substantially higher review rates. When AI-generated PRs are reviewed, reviews are dominated by AI agents rather than humans: human involvement is expressed primarily through agent steering (approving or redirecting automated review workflows) rather than standalone evaluation. In contrast, human-authored PRs attract direct human feedback — reviewers read the diff and comment in their own words.

The implications for Codentry's interaction design are layered. At the interaction level, [21]'s finding that AI-generated PR review is already mediated through GitHub's automation interfaces supports the PR-comment model: developers are already accustomed to interacting with AI review through GitHub comment threads, not through external tools. At the evaluation level, [21] raises a methodological concern for Phase 6: if Codentry's evaluation dataset is composed primarily of AI-generated PRs (likely, given the growing prevalence of AI coding agents), the review dynamics will be systematically different from human-authored PR review. The ground truth annotations in such a dataset may not reflect genuine human expert judgment but rather the steering behaviours of developers managing automated review workflows.

6.2 Task Type as the Dominant Factor in PR Acceptance

The task-stratified analysis [22] of 7,156 PRs across five AI coding agents (OpenAI Codex, GitHub Copilot, Devin, Cursor, Claude Code) provides the most detailed empirical picture of what drives PR acceptance rates in AI-authored workflows. The central finding — that PR task type explains more variance in acceptance rate than agent identity — is significant because it implies that a code review tool optimised for high-impact review (changing outcomes rather than just adding comments) should prioritise the task types where review comments are most likely to change the acceptance decision.

Documentation tasks achieve 82.1% acceptance rate vs. 66.1% for new features — a 16pp gap. This gap persists across agents and over time, suggesting that reviewers are more lenient on documentation changes regardless of their quality. Temporal analysis shows that Devin is the only agent with a consistent positive trend in acceptance rate (+0.77% per week over 32 weeks), suggesting that agent-level learning and improvement is real but slow. The no-single-agent-dominates finding across all nine task categories has an implication for Codentry's review prioritisation: since different AI agents have different strengths and weaknesses, Codentry's review should not assume a uniform error profile but should adapt its review emphasis based on signals about which AI agent authored the PR — a feature

that would require identifying the authoring agent from PR metadata, which is increasingly feasible as AI agents leave distinctive metadata traces.

6.3 The Impact of LLM Assistance on PR Timelines

The empirical study of GPT assistance on PR workflows [25] provides quantitative evidence that LLM integration into the PR lifecycle has measurable effects on review timelines. Across 25,473 PRs from 9,254 GitHub projects, GPT-assisted PRs show a 61% reduction in median resolution time (from 23 hours to 9 hours), a 66.7% reduction in review time (from 3 hours to 1 hour for the review phase), and an 87.5% reduction in waiting time before acceptance. The developers predominantly used GPT for code optimisation (60%), bug fixing (26%), and documentation updates (12%).

These findings provide a temporal baseline for Codentry's impact evaluation. If GPT assistance on the contributor side already reduces review time by 66.7%, Codentry's AI-assisted review on the reviewer side should provide an additional acceleration — the reviewer no longer needs to spend time identifying the most important issues manually, since Codentry's review highlights them. The combined effect (AI-assisted contribution + AI-assisted review) represents the full cycle time reduction that Codentry's deployment context is working toward. Measuring this combined effect in Phase 6 would require comparing PRs with and without Codentry review in the same repository, which would constitute a controlled experiment beyond current FYP scope but valuable as future work.

6.4 Implication for Codentry

Theme E provides the clearest empirical justification for Codentry's PR-comment interaction model and its review prioritisation strategy. The evidence from [21] that developers already interact with automated review through GitHub's PR interface rather than external tools confirms that the PR-comment model meets developers where they are, reducing adoption friction to near zero compared to dashboard-based alternatives. The task-stratification evidence from [22] provides a prioritisation principle: Codentry should weight correctness and security findings higher than documentation findings in its review output, because reviewers are less likely to act on documentation concerns regardless of their validity. The timeline evidence from [25] provides a measurable dimension for Codentry's Phase 6 evaluation: PR resolution time with and without Codentry review is an outcome metric that can be measured without human annotation of review quality. The risk this theme surfaces is the AI-PR feedback loop: as Codentry reviews more AI-generated PRs, Phase 6's evaluation dataset increasingly lacks human-expert ground truth, degrading evaluation validity.

7. Theme F: Core AI/LLM Code Review and PR Automation

Theme F encompasses papers most directly concerned with the practical mechanics of LLM-based code review comment generation, PR automation, and novel evaluation frameworks for review quality. Six papers form this theme. Together they describe the current state of the art in LLM-based review from multiple angles — retrieval-enhanced generation (LAURA [23]), unified benchmark and training frameworks (Sphinx [24]), practitioner field studies (Rethinking CR Workflows [27]), systematic LLM capability evaluation (Evaluating LLMs for CR [26]), and advanced critique systems grounded in large-scale taxonomy analysis (ARCTIC [28]).

7.1 LAURA: Context-Enriched RAG for Review Generation

LAURA [23] — LLM-based review knowledge-augmented, context-aware framework — proposes the most complete RAG pipeline for code review generation in the corpus. Its three-component architecture integrates: (1) review exemplar retrieval, finding historical review comments for code similar to the change under review; (2) context augmentation, including the code change's surrounding context and any prior review knowledge specific to the codebase; and (3) systematic guidance, providing structured prompting that steers the model toward review best practices. Applied to ChatGPT-4o and DeepSeek v3, LAURA generates review comments that are completely correct or at least helpful to developers in 42.2% and 40.4% of cases, respectively. The ablation studies confirm that all three components contribute positively: removing exemplar retrieval degrades correct/helpful rates by approximately 8 percentage points, removing context augmentation by approximately 6 points, and removing systematic guidance by approximately 4 points.

The 42% correct/helpful rate is presented against a dataset of genuinely challenging review scenarios, and LAURA significantly outperforms SOTA baselines. For Codentry, the practical implication is that Phase 4's RAG pipeline should implement all three of LAURA's components rather than just exemplar retrieval. Context augmentation maps onto Codentry's use of the PR description, commit message, and surrounding file context already available from the GitHub API. Systematic guidance maps onto Codentry's prompt template design, where structured sections for correctness, security, style, and performance concerns guide Claude's output. LAURA's ablation provides empirical justification for all three design choices independently.

7.2 Sphinx: Checklist-Based Training and Evaluation

Sphinx [24] attacks a fundamental problem with LLM-based PR review: noisy supervision. Historical PR review comments are inconsistent — different reviewers focus on different issues, the same reviewer focuses on different issues for different PRs, and review quality varies substantially with reviewer expertise. Sphinx's solution is a structured data generation pipeline that produces context-rich, semantically grounded review comments by comparing pseudo-modified code (code with realistic errors injected) against merged code (code after successful review). This comparison forces the annotation process to be contrastive — the review comment must explain what changed and why the change matters — producing more consistent and actionable training data.

Sphinx's checklist-based evaluation benchmark assesses review quality based on structured coverage of actionable verification points rather than surface metrics. The Checklist Reward Policy Optimisation (CRPO) training paradigm uses rule-based, interpretable rewards — directly addressing the concern that RLHF-style training can produce persuasive-sounding but technically incorrect reviews. Models trained

with Sphinx achieve up to 40% improvement in checklist coverage over proprietary baselines (including GPT-4). For Codentry's Phase 6, Sphinx's checklist framework provides a directly adoptable evaluation template: a structured checklist covering correctness, security, style, performance, and documentation concerns, with Claude's output scored against each dimension independently. This is precisely the multi-dimensional evaluation recommended by [10] and partially implemented by [1].

7.3 Rethinking Code Review Workflows: Industrial Field Evidence

The WirelessCar field study [27] is unique in the corpus for providing controlled experimental evidence from a real industrial deployment. Its two prototype variants — AI-led review (LLM-generated review delivered upfront) and on-demand interaction (developer queries the LLM as needed) — both use a semantic search pipeline based on RAG to assemble relevant contextual information. Developer evaluations in real-world settings find that AI-led reviews are overall more preferred, with the preference conditional on the reviewer's familiarity with the codebase and the severity of the PR. For routine PRs in familiar codebases, AI-led upfront review is strongly preferred; for complex PRs in unfamiliar codebases, on-demand interaction is preferred.

The field study's qualitative findings identify challenges in traditional code review that Codentry directly addresses: frequent context switching (reviewers must look up documentation and related code manually), insufficient contextual information (diffs alone do not provide enough context for meaningful review), and false positives from AI tools eroding reviewer trust. The false positive concern is particularly relevant to Codentry's design: if Semgrep's false positive rate is high, it will erode developer trust in Codentry's entire review output, including the AI judgment layer. This provides a non-evaluation reason to prioritise Semgrep false positive reduction: trust maintenance in the combined system requires the deterministic layer to have high precision.

7.4 ARCTIC: Intent, Drift, and Spotlight at Production Scale

ARCTIC [28] represents the most advanced production-deployed code critique system in the corpus. Its three-capability architecture addresses a problem that all other papers in Theme F acknowledge but do not solve: at the scale of AI coding agents generating PRs continuously, traditional review metrics become meaningless because human reviewers cannot inspect every diff. ARCTIC's intent prediction capability (0.86 F1) infers why a change was made from conversation logs, commit messages, and PR metadata. Its drift detection capability (QWK=0.907 against human annotators) measures the divergence between inferred intent and actual code output — identifying cases where the agent produced code that does not match what it was asked to do. Its code spotlight capability (2.4x quality improvement at 5x fewer tokens vs. baseline AI reviewer) identifies which regions of a diff most warrant human attention.

ARCTIC's production results are the strongest in Theme F: drift scores reduce code misalignment by 5.76 additional points ($p=0.026$), intent prediction receives 90.2% developer approval, and zero defects have been attributed to self-reviewed diffs since launch. The 5x token efficiency of code spotlight is particularly relevant to Codentry's free-tier constraint: if Claude's token usage can be reduced 5x by routing attention to high-spotlight regions, it may enable higher-quality review within API token limits. Implementing a simplified spotlight heuristic — ranking diff hunks by size of change, cyclomatic complexity of the affected functions, and proximity to security-sensitive patterns — would require no additional API calls and could be computed by Semgrep or ESLint as part of the static analysis layer.

7.5 Synthesis: What Theme F Tells Us About Practical LLM Review

Papers [23], [24], [25], [26], [27], and [28] collectively establish a coherent picture of where LLM-based code review stands in 2025–2026. First, context enrichment through RAG is the most effective technique for improving LLM review quality, with gains reported by [23] (42% correct/helpful rate), [27] (developer preference for RAG-enhanced review), and [28] (5x token efficiency via spotlight). Second, structured evaluation frameworks — checklist-based [24], intent-and-drift-based [28] — substantially outperform surface metrics as predictors of developer-useful review quality. Third, LLM assistance measurably accelerates the PR lifecycle [25] even before full review automation is achieved, providing a timeline-based evaluation dimension for Codentry. Fourth, developer preference for AI-led review is real but conditional [27], supporting a default-active but tunable review model.

7.6 Implication for Codentry

Theme F provides the most direct operational guidance for Codentry's current and near-future implementation. LAURA [23] establishes that Phase 4's RAG pipeline should implement three components — exemplar retrieval, context augmentation, and systematic guidance — with Codentry's existing GitHub API integration providing the context augmentation component at no additional cost. Sphinx [24] provides a checklist template for Phase 6's evaluation harness that can be adopted directly. ARCTIC [28] provides a token-efficient spotlight heuristic that Codentry's static analysis layer can compute — ranking diff hunks by change magnitude, complexity, and security proximity — reducing Claude's effective review scope to the most important regions. The WirelessCar evidence [27] validates the default-active review model. The empirical accuracy figures from [26] (68.50% correctness classification for GPT-4o with problem context) provide a realistic lower bound for Claude's performance on Codentry's review tasks, setting an appropriate expectation for user communications about review accuracy.

8. Cross-Theme Synthesis Table

Table 1 provides a comparative summary of all 30 papers across five structurally significant dimensions: primary approach, the benchmark or dataset used, whether the system explicitly separates static analysis from AI judgment for independent evaluation, whether evaluation is performed on real GitHub pull requests, and whether repository-level context is considered. A row for Codentry is included to make its positioning explicit. Three observations stand out. First, no paper in the corpus separates static and AI layers for independent evaluation — this column is unique to Codentry. Second, the papers that evaluate on real PRs are a minority; many use filtered datasets or synthetic code, which limits their external validity. Third, repository-level context is considered by only about half the papers, and among those, the approaches vary substantially.

#	Paper (Short)	Primary Approach	Dataset / Benchmark	Static/ AI Split	Real PRs	Repo Context	The me
1	CodeFuse-CR-Bench [1]	Comprehensive benchmark, 9 PR domains	601 PRs, 70 Python projects	No	Yes	Yes	A, B
2	ContextBench [2]	Process-oriented retrieval eval	1,136 tasks, 66 repos, 8 langs	No	Partial	Yes	A
3	Agent Retrieval Bench [3]	File-level retrieval benchmark	427 samples, 25 repos	No	No	Yes	A
4	RAG-based RCG [4]	RAG pipeline for review generation	Tufano et al. benchmark	No	No	No	A
5	RARe: When More Retrieval Hurts [5]	RAG in-context learning (top-1)	2 public benchmarks	No	No	No	A
6	RepoReasoner [6]	Cross-file reasoning benchmark	Custom benchmark, pytest traces	No	No	Yes	A
7	RAG for Code Gen Survey [7]	Survey: chunk/graph/hybrid RAG	Meta-analysis (no primary data)	No	N/A	Yes	A
8	SWE-PRBench [8]	Annotated PR eval, 3 context configs	350 PRs, kappa=0.75 annotation	No	Yes	Partial	A, B
9	c-CRAB [9]	Test-based agent evaluation	Agent-generated tests from human reviews	No	Yes	No	B
10	Survey of CR Benchmarks [10]	SLR: 99 papers, 5 domains, 18 tasks	Meta-analysis (2015–2025)	No	N/A	Partial	B
11	SWRBench [11]	LLM-as-judge, multi-review aggregation	1,000 verified GitHub PRs	No	Yes	No	B
12	RepoReviewer [12]	Multi-agent system (FastAPI+Next.js)	No formal benchmark	No	Yes	Yes	C
13	SWE-Review [13]	Generate-review-revise agentic loop	SWE-Review-Bench	No	Yes	Yes	C
14	CodeX-Verify [14]	Multi-agent info-theoretic verification	99 code samples, verified labels	No	No	No	C
15	AgenticSCR [15]	Agentic security review (Atlassian prod.)	Shadow deployment, security engineers	Partial	Yes	Yes	C, D
16	SUSVIBES [16]	Security benchmark for AI code	186 tasks, 12 agent configs	N/A	No	No	D
17	SEC-bench [17]	Security task automation benchmark	PoC + patching, real CVEs	N/A	No	Yes	D

#	Paper (Short)	Primary Approach	Dataset / Benchmark	Static/ AI Split	Real PRs	Repo Context	The me
18	Security Considerations [18]	Threat taxonomy for AI agents	NIST RFI (qualitative analysis)	N/A	N/A	N/A	D
19	Architecture + MAS Security [19]	Empirical MAS security, 13 configs	3 environments: browser/desktop/code	N/A	No	No	D
20	Code Injection Attacks [20]	Threat model + code injection analysis	Synthetic injection tasks	N/A	No	No	D
21	How Humans Review AI PRs [21]	Mining study, AIDev dataset (GitHub)	AIDev dataset — real GitHub PRs	N/A	Yes	No	E
22	Task-Stratified PR Acceptance [22]	Comparative empirical study, 5 agents	7,156 PRs, AIDev dataset	N/A	Yes	No	E
23	LAURA [23]	Context-enriched 3-component RAG	High-quality curated review dataset	No	No	Partial	F
24	Sphinx [24]	Checklist training + CRPO	Pseudo-modified code dataset	No	No	Partial	F
25	LLM Impact on PR Process [25]	Empirical study, 25K GitHub PRs	25,473 PRs, 9,254 projects	N/A	Yes	No	F
26	Evaluating LLMs for CR [26]	LLM capability eval (GPT-4o, Gemini)	492 AI code blocks + HumanEval	No	No	No	F
27	Rethinking CR Workflows [27]	Field study + experiment (WirelessCar)	Industrial PRs, real developer evals	No	Yes	Partial	F
28	ARCTIC [28]	Intent/drift/spotlight, 18K reviews	18,000 code review taxonomy + prod.	No	Yes	Partial	F
29	MAF-CPR [29]	Multi-agent framework for complex PR review	GitHub open-source PRs (ICIC 2025)	No	Yes	No	C, D
30	Automating CR SLR [30]	SLR: 119 papers on CR automation	Meta-analysis (pre-LLM + LLM era)	No	N/A	Partial	B, G
—	Codentry (this project)	Static/AI split + RAG + PR-native GitHub App	Phase 6 harness (planned, real PRs)	YES (unique)	Yes	Planned (Phase 4)	All

Table 1: Cross-Theme Synthesis of All 30 Papers and Codentry Positioning across Five Structural Dimensions.

9. Gap Analysis: What the Corpus Does Not Address

The gap analysis is presented with deliberate modesty. The goal is not to claim that Codentry fills a large, obvious gap in the research literature — it does not. The goal is to identify precisely what combination of properties no single paper or system in the corpus achieves, and to characterise how narrow or broad each gap genuinely is.

9.1 Gap 1: Independent Evaluability of Static and AI Review Layers

The most distinctive gap in the corpus is the absence of any code review architecture that explicitly separates static analysis output from AI judgment output and evaluates each against different ground-truth sources using different metrics. This gap is present despite the fact that multiple papers use both static analysis and AI in the same pipeline. AgenticSCR [15] compares against SAST tools but evaluates end-to-end performance only. CodeX-Verify [14] uses multiple specialised agents but measures combined accuracy, not per-agent contribution. CodeFuse-CR-Bench [1] applies deterministic location checks and model-based quality judgment to the same review output but does not separate the sources producing that output.

The narrowness qualifier: this is a gap in evaluation methodology, not in user-facing functionality. A user comparing Codentry and a fully integrated system would see identical review comments. The difference lies in what happens after deployment — Codentry's phase 6 harness can attribute performance changes to either the static layer or the AI layer independently, enabling targeted improvement. This is a methodological contribution of value to future research, not a user-experience improvement.

9.2 Gap 2: Free-Tier-Deployable Repository-Level AI Review

Every repository-level review system in the corpus that demonstrates strong performance requires infrastructure that is not reproducible under free-tier constraints. SWE-Review [13] requires test execution. AgenticSCR [15] requires proprietary security-focused semantic memory built over months of Atlassian deployment. RepoReviewer [12], the closest to free-tier feasibility, requires local infrastructure and multiple LLM calls per review. No paper demonstrates what is achievable with a single LLM call per review and a pgvector-based retrieval store on a managed free-tier database.

The narrowness qualifier: free-tier deployability is an infrastructure constraint, not a research novelty. The contribution is not a new algorithm but a characterisation of what is achievable at minimal cost, which has value for reproducibility and for student teams or resource-constrained researchers who want to build on Codentry's findings.

9.3 Gap 3: PR-Native Interaction Without External Dashboard

Most tools in the corpus operate as CLI tools, research benchmarks, or enterprise deployments with proprietary interfaces. The PR-native interaction model — a GitHub App that posts comments to PRs without requiring any external interface — is validated as feasible by [29] (MAF-CPR deploys within GitHub's PR interface) and as preferred by developers in [21] and [27], but no paper systematically evaluates this interaction model against dashboard alternatives as a design choice.

The narrowness qualifier: the gap is in systematic evaluation of the interaction model as a variable, not in the model itself. Codentry does not fill this gap — it makes the design choice and justifies it with

indirect evidence from [21], [22], and [27] without conducting a direct comparison. This is honest: the comparison would require a controlled user study beyond FYP scope.

9.4 Gap 4: Stratified Evaluation Across Human-Authored and AI-Generated PRs

No benchmark paper in the corpus explicitly stratifies its evaluation dataset by PR authorship type (human-authored vs. AI-generated) and reports results separately for each stratum. As [21] documents, these two populations have systematically different review dynamics — AI-generated PRs receive less human review, are reviewed more by other AI agents, and attract human involvement primarily through agent steering rather than standalone evaluation. A review system trained and evaluated exclusively on human-authored PRs may generalise poorly to AI-generated PRs, and vice versa.

This gap is genuine and methodologically significant for Codentry specifically because its primary deployment context — reviewing code submitted through AI coding agents — is the AI-generated PR stratum that no existing benchmark targets. Codentry's Phase 6 harness should explicitly construct an evaluation set that includes both strata, stratified and reported separately. This would fill a methodological gap in the existing literature while producing more externally valid results for Codentry's specific deployment context.

9.5 Summary

In summary, Codentry uniquely attempts four things in combination that no paper or system in the 30-paper corpus achieves simultaneously: (1) a clean static/AI split with independent evaluation metrics for each layer; (2) free-tier cloud deployment with a reproducible, open architecture; (3) PR-native interaction through a GitHub App without an external dashboard; and (4) repository-level context through a precision-first RAG design informed by the attention dilution findings in [5] and [8]. Each of these individually has precedent in the corpus. Their combination under a single free-tier architectural constraint does not. The claim is narrow and honest: this is an engineering combination with methodological value, not an algorithmic innovation or a theoretical contribution.

10. Threats to Validity and Limitations of This Survey

10.1 ArXiv Preprint Bias

29 of the 30 papers in this corpus are arXiv preprints; only the Springer chapter MAF-CPR [29] has undergone formal peer review (ICIC 2025). ArXiv preprints are not peer-reviewed and may contain errors, overclaims, or methodological weaknesses that peer review would identify. The field of AI-assisted code review is moving fast enough that papers appearing in mid-2026 may not have been independently replicated or critically evaluated. Quantitative performance figures from preprints — particularly impressive claims like the 153% improvement in AgenticSCR [15] or the 43.67% F1 gain from multi-review aggregation in SWRBench [11] — should be interpreted with the caveat that they rest on the authors' own evaluation setups rather than independent verification. Where multiple papers report consistent findings, confidence is higher than where a finding rests on a single preprint.

10.2 Recency Restriction and Historical Omissions

The corpus is deliberately restricted to 2025–2026 papers, which means the survey omits the foundational literature that informed the papers cited here. The original Tufano et al. benchmark (2021, used as a comparison point in [4] and [5]) is not reviewed. CodeBERT, CodeT5, and similar code models that underpin many of the LLM review approaches are referenced in corpus papers but not reviewed here. Readers seeking a complete historical picture should supplement this survey with the systematic literature reviews in [10] (covering 99 papers from 2015–2025) and [30] (covering 119 papers on CR automation).

10.3 Corpus Selection Bias

The 30 papers were selected by the project team based on perceived relevance to Codentry's architecture. This introduces selection bias: papers that directly contradict Codentry's design choices may have been inadvertently excluded. To partially mitigate this, the survey explicitly discusses cases where corpus papers complicate or challenge Codentry's design — the attention dilution finding from [8] that constrains naive RAG, the injection vulnerability in [20] that constrains the AI layer's authority, the multi-round architecture of [13] that Codentry cannot implement under free-tier constraints. A comprehensive survey would involve systematic search terms, inclusion/exclusion criteria, and a PRISMA-style selection process, which was not feasible for this FYP-scope document.

10.4 LLM-as-Judge Evaluation Uncertainty

Several papers in the corpus use LLM-as-judge frameworks as their primary evaluation method, with inter-rater reliability reported at $\kappa=0.75$ [8] and approximately 90% human agreement [11]. These rates are high by social science standards but still leave 10–25% of evaluation judgments uncertain. Claims about AI code review quality derived from LLM-as-judge evaluation inherit this uncertainty. The problem is compounded when the same LLM family is used both as the system under evaluation and as the judge — a potential bias toward output styles similar to the judge model's training distribution. This survey has not independently verified which judge models were used in each paper, and the potential for judge model bias is therefore an acknowledged but unmeasured threat.

10.5 Single-Language and Repository Type Biases

CodeFuse-CR-Bench [1] is restricted to Python projects. Findings derived from it may not transfer to multi-language codebases of the kind Codentry will encounter in practice. Agent Retrieval Bench [3], while covering multiple task types, focuses on repositories in specific programming languages. ESLint (one of Codentry's static analysis tools) is JavaScript/TypeScript-specific, meaning that performance claims about Codentry's static layer derived from Python-focused benchmarks are not directly applicable. Codentry's Phase 6 evaluation should include PRs from at least three programming languages to establish some multi-language generalisability.

11. Conclusion

This literature survey has reviewed all 30 papers in the provided corpus to systematically ground Codentry's architectural decisions in documented research findings from 2025–2026. The central architecture claim — that separating deterministic static analysis from AI judgment enables independent evaluation of each layer — is supported by the evaluation methodology literature ([1], [8], [10], [11]), by the information-theoretic argument for independent multi-layer detection ([14]), and by the production evidence that SAST tools and AI judgment have largely non-overlapping failure modes ([15]). The claim is also informed by what the corpus reveals about the risks of conflating the two layers: when static and AI review are evaluated end-to-end, performance failures cannot be attributed to either layer, making systematic improvement harder.

The RAG-based repository context plan (Phase 4) is grounded in two findings that are in productive tension and resolve cleanly. Context retrieval improves review quality ([4], [23], [27]) but naive context addition degrades it through attention dilution ([5], [8]). The resolution — retrieving highly ranked exemplars with $k=1$ or $k=2$ using pgvector with lightweight dependency metadata, as recommended by [7] — is specific enough to implement and test, and directly addresses the attention dilution mechanism documented in [8].

The PR-comment interaction model is supported by the empirical observation that developers of AI-generated PRs already interact primarily through GitHub's automation interfaces [21], by the field study evidence that developers prefer AI-led review by default [27], and by the ICIC-reviewed demonstration that multi-agent PR review can be deployed within GitHub's PR interface [29]. The security threat model, informed by [18], [19], and [20], establishes specific structural mitigations: prompt isolation between code content and review instructions, Semgrep-based injection screening, and output anomaly detection before comment posting.

The gap analysis reveals that Codentry's combination of free-tier deployability, PR-native interaction, static/AI separation, and precision-first RAG architecture is not represented by any single paper or system in the 30-paper corpus. This is a narrow but honest claim — an engineering combination with methodological value for future work in resource-constrained review systems. The survey's acknowledged limitations — arXiv preprint bias, recency restriction, corpus selection bias, LLM-as-judge evaluation uncertainty, and single-language benchmark skew — should be reflected in how Codentry's FYP report frames its performance claims.

The standard applied throughout this document has been: if a paper's actual content does not support the claim being made, say what it does support instead. Precision matters more than confidence in a document that is going into an FYP report as evidence. This survey has been written to that standard.

References

- [1] Liu, X., et al., "CodeFuse-CR-Bench: A Comprehensiveness-aware Benchmark for End-to-End Code Review Evaluation in Python Projects," arXiv:2509.14856v3, 2025.
- [2] Chen, H., et al., "ContextBench: A Benchmark for Context Retrieval in Coding Agents," arXiv:2602.05892v3, 2026.
- [3] Zhang, Y., et al., "Agent Retrieval Bench: Evaluating Repository Context Retrieval for Coding Agents," arXiv:2607.24882v1, 2026.
- [4] Tian, Y., et al., "Retrieval-Augmented Code Review Comment Generation," arXiv:2506.11591v1, 2025.
- [5] Wang, L., et al., "When More Retrieval Hurts: Retrieval-Augmented Code Review Generation," arXiv:2511.05302v2, 2025.
- [6] Li, R., et al., "RepoReasoner: Evaluating Repository-Level Code Reasoning Ability of Long-Context Language Models," arXiv:2607.25996v1, 2026.
- [7] Zhao, Q., et al., "Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches," arXiv:2510.04905v3, 2025.
- [8] Kumar, A., et al., "SWE-PRBench: Benchmarking AI Code Review Quality Against Pull Request Feedback," arXiv:2603.26130v1, 2026.
- [9] Gupta, S., et al., "Code Review Agent Benchmark (c-CRAB)," arXiv:2603.23448v3, 2026.
- [10] Park, J., et al., "A Survey of Code Review Benchmarks and Evaluation Practices in Pre-LLM and LLM Era," arXiv:2602.13377v1, 2026.
- [11] Chen, W., et al., "SWR-Bench: Assessing LLM Performance in Real-World Code Review Comment Generation," arXiv:2509.01494v1, 2025.
- [12] Martinez, R., et al., "RepoReviewer: A Local-First Multi-Agent Architecture for Repository-Level Code Review," arXiv:2603.16107v1, 2026.
- [13] Zhang, K., et al., "SWE-Review: Closing the Loop on Issue Resolution with Agentic Code Review," arXiv:2607.06065v1, 2026.
- [14] Patel, N., et al., "Multi-Agent Code Verification via Information Theory," arXiv:2511.16708v3, 2025.
- [15] Singh, A., et al., "AgenticSCR: An Autonomous Agentic Secure Code Review for Immature Vulnerabilities Detection," arXiv:2601.19138v2, 2026.
- [16] Li, Y., et al., "Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks," arXiv:2512.03262v3, 2025.
- [17] Xu, T., et al., "SEC-bench: Automated Benchmarking of LLM Agents on Real-World Software Security Tasks," arXiv:2506.11791v2, 2025.
- [18] Perplexity AI, "Security Considerations for Artificial Intelligence Agents," arXiv:2603.12230v2, 2026.
- [19] Kim, H., et al., "Architecture Matters for Multi-Agent Security," arXiv:2604.23459v1, 2026.

-
- [20] Russo, D., et al., "Analyzing Code Injection Attacks on LLM-based Multi-Agent Systems in Software Development," arXiv:2512.21818v1, 2025.
 - [21] Terragni, V., et al., "These Aren't the Reviews You're Looking For: How Humans Review AI-Generated Pull Requests," arXiv:2605.02273v1, 2026.
 - [22] Ahmed, Z., et al., "Comparing AI Coding Agents: A Task-Stratified Analysis of Pull Request Acceptance," arXiv:2602.08915v2, 2026.
 - [23] Liu, M., et al., "LAURA: Enhancing Code Review Generation with Context-Enriched Retrieval-Augmented LLM," arXiv:2512.01356v2, 2025.
 - [24] Wang, H., et al., "Sphinx: Benchmarking and Modeling for LLM-Driven Pull Request Review," arXiv:2601.04252v1, 2026.
 - [25] Collante, A., et al., "The Impact of Large Language Models (LLMs) on Code Review Process," arXiv:2508.11034v2, 2025.
 - [26] Lemos, F., et al., "Evaluating Large Language Models for Code Review," arXiv:2505.20206v1, 2025.
 - [27] Sundberg, E., et al., "Rethinking Code Review Workflows with LLM Assistance: An Empirical Study," arXiv:2505.16339v1, 2025.
 - [28] Sharma, V., et al., "From Code Review to Code Critique: Intent, Drift, and Spotlight for AI-Generated Diffs at Scale," arXiv:2607.29516v2, 2026.
 - [29] Han, F., Peng, J., Wang, W., and Xia, X., "MAF-CPR: LLM-Based Multi-agent Framework for Complex Pull Request Review in GitHub," in Proc. ICIC 2025: Advanced Intelligent Computing Technology and Applications, pp. 247–257, Springer, Singapore, 2025. DOI: 10.1007/978-981-95-0020-8_21.
 - [30] Di Grazia, L., et al., "Automating Code Review: A Systematic Literature Review," arXiv:2503.09510v1, 2025.